

ServiceNow Project Documentation Format

Implement Client Script & UI Policy (Incident)

1. Introduction

Project Title: Implement Client Script & UI Policy (Incident)

Team Members: Vishnu Raj, Lokeshwaran, Sandeep, Gowtham ,Arun Allwin Raja. **Role:** ServiceNow System Administration / Configuration Team.

2. Project Overview

Purpose: The project demonstrates how ServiceNow UI Policies and Client Scripts work together to control Incident form behaviour, automate field updates, validate submissions and prevent inappropriate list editing.

Key Features:

- High Impact UI Policy — Assignment group becomes mandatory and Urgency becomes read-only when Impact is 1 - High.
- onChange Client Script — automatically sets Urgency to 1 - High when Impact becomes High.
- onSubmit Client Script — prevents saving a High Impact Incident when Assigned To is empty.
- onCellEdit Client Script — prevents direct State changes through Incident list editing.
- Reverse condition — High Impact UI Policy actions are reversed when the Impact condition becomes false.

3. Architecture

Platform: ServiceNow Incident [incident] table with browser-based ServiceNow UI, UI Policy configuration, Client Scripts and native reference fields.

Application logic: UI Policy controls dynamic field state; onChange handles Impact-driven Urgency automation; onSubmit validates save conditions; onCellEdit controls State changes from the list.

Data: Incident records and related reference data are stored and managed within the ServiceNow instance. No separate MongoDB, external API or machine-learning model is required.

4. Setup Instructions

Prerequisites: ServiceNow instance access, configuration permissions and a modern web browser.

Installation / Configuration: Open the ServiceNow instance and create/activate the Incident UI Policy and Client Script records described in the project.

5. Configuration Structure

- Milestone 1 — High Impact Control UI Policy.
- Milestone 2 — Urgency UI Policy Action.
- Milestone 3 — Auto set urgency for high impact onChange Client Script.
- Milestone 4 — Prevent save if Assigned To missing onSubmit Client Script.
- Milestone 5 — Prevent State change via list edit onCellEdit Client Script.
- Milestone 6 — Functional testing activities and evidence.

6. Running the Application

There is no separate frontend/backend server to start. The solution runs inside the ServiceNow instance through the Incident form and Incident list.

7. API / Integration Documentation

No external REST API is required. The solution uses native ServiceNow client-side APIs such as g_form within Client Scripts to read and update Incident fields and display messages.

8. Authentication & Authorization

Authentication is handled by the ServiceNow instance login/session mechanism. Authorization is governed by ServiceNow roles and access controls. Client Scripts and UI Policies provide interface behaviour and are not a replacement for server-side access control.

9. User Interface

The interface is the standard ServiceNow Incident form and Incident list. Field behaviour changes dynamically according to Impact and clear messages are shown when an action is blocked or an automatic update occurs.

- High Impact: Assignment group shows mandatory behaviour and Urgency becomes read-only.
- High Impact change: Urgency is automatically set to High.
- Invalid High Impact submission: Assigned To validation is displayed and save is blocked.
- List State edit: an alert informs the user to open the Incident and the edit is blocked.

10. Testing

Testing was performed through Milestone 6 functional test activities.

- Activity 1 — Mandatory Enforcement.
- Activity 2 — Successful Save.
- Activity 3 — Reverse Condition.
- Activity 4 — List Edit Blocking.
- Activity 5 — Form-Based Update.

11. Screenshots / Demo

Project evidence consists of screenshots captured from the ServiceNow instance for UI Policy configuration, Client Script configuration and Milestone 6 testing.

12. Known Issues

- Scope is limited to the Incident form/list behaviour configured in the ServiceNow instance.
- Client-side validation is not a substitute for server-side security controls.
- No external performance-monitoring or API integration is included.

13. Future Enhancements

- Add server-side validation where required.
- Extend dynamic rules to additional Incident fields.
- Add automated regression testing.
- Add dashboards/reporting.
- Add notifications or workflow automation for selected Incident conditions.